

FlowHub

Eine KI-gestützte persönliche Inbox

Andreas Imboden · CAS AI-Assisted Software Engineering · FFHS FS26

Das Projekt — und die Erfahrung, es mit KI zu bauen

Das Problem — und die Idee

Der digitale Alltag produziert ständig Schnipsel: ein Film, ein Artikel, das Foto einer Quittung.

Heute

Idee → welche App? → öffnen → kategorisieren → ablegen

5+ Schritte — oft vergessen

Mit FlowHub

Idee → **Telegram** → fertig

1 Schritt — die KI übernimmt

Ein **Telegram-Bot** als einziger Eingang: FlowHub **erkennt** den Input, **kategorisiert** ihn und **routet** ihn automatisch an den richtigen Self-Hosted-Service.

Konkret: das Skill-System

Jeder Input wird einem **Skill** zugewiesen — Erkennung über Keywords, URL-Muster und, wenn nötig, ein **LLM**.

Input	Skill	landet in
heise.de-Artikel	ArticleSkill	Wallabag (read-later) ✓
„The Imitation Game“, share.google/...	MovieSkill	Vikunja (Watchlist) ✓
nichts passt	GenericSkill	Inbox (PostgreSQL) ✓
jellyfin.org „ausprobieren“	HomelabSkill	Vikunja Kanban · geplant
Foto einer Quittung	DocumentSkill	paperless-ngx (DMS) · geplant



heute live · geplant = Roadmap (gleiche `ISkillIntegration`-Schnittstelle).

Unklar? → Der Bot **fragt mit 2-3 Optionen zurück** (Confidence-Score).

Unter der Haube: **Klassifikation** → **Anreicherung** → **Routing**

Zwei entkoppelte MassTransit-Consumer (ADR 0003), verbunden über Events:

- ① CaptureCreated –
 - ▶ CaptureEnrichmentConsumer
 - IClassifier → AiClassifier — LLM, structured JSON — Fallback
 - ▶ KeywordClassifier
 - Ergebnis: matched_skill · project · title · entities · tags
 - EnricherDispatcher → IEnricher[project]
 - ↳ ZitateEnricher — 2. LLM-Call: kurze Autor-Info
- ② CaptureClassified –
 - ▶ SkillRoutingConsumer
 - ISkillIntegration[Name] → VikunjaSkillIntegration
 - ↳ PUT /api/v1/projects/{Zitate}/tasks → ③ Completed (= Vikunja-Task-ID)

Stufe 1 klassifiziert (LLM, mit Keyword-Fallback) **und** reichert an; Stufe 2 routet zum Ziel-Dienst. Enricher leben in `FlowHub.AI/Enrichers` — **pro Bucket eine IEnricher-Klasse**, nur für Vikunja. **Zwei System-Prompts** ans LLM.

Beispiel: Capture ist nur das Zitat

Eingang & Verarbeitung

Telegram-Capture:

```
Talk is cheap. Show me the code.
```

→ **AiClassifier** (LLM) erkennt das Zitat:

```
matched_skill: Vikunja · project: Zitate
```

```
entities { quote, author: "Linus  
Torvalds" }
```

→ **ZitateEnricher** (2. LLM-Call) → kurze
Autor-Info

→ **VikunjaSkillIntegration** → PUT
.../tasks → **Completed**

Ergebnis — Vikunja-Task im Projekt „Zitate“

*"Talk is cheap. Show me the code." —
Linus Torvalds*

*About Linus Torvalds: <2-4-Satz-Info
vom Modell>*

Den **Autor** liefert schon die Klassifikation (Modell-Wissen); die **Autor-Info** ergänzt der Enricher mit „Never invent facts.“ als Halluzinations-Stopper.

System-Prompt ① — Klassifikation (Zitat)

`AiClassifier` → `IChatClient.GetResponseAsync<AiClassificationResponse>` · role: system (`AiPrompts`)

You classify user-captured snippets for a personal knowledge tool called FlowHub.

For each capture, return:

- tags: 1–5 short lowercase tags describing the snippet
- matched_skill: which downstream skill should handle it. Choose exactly ONE:
 - "Wallabag" – the snippet is a URL or article worth saving for later reading
 - "Vikunja" – the snippet is a task, todo, OR a structured piece of content that belongs in a Vikunja project (quote, movie, book, ...)
 - " " – none of the above; it will be marked as Orphan
- project: when matched_skill="Vikunja", pick the best matching project from this list. If unsure, pick "Inbox".
 - Available: Inbox, Zitate
 - Leave empty otherwise.
- title: a 3–8 word title summarising the snippet (omit only if the snippet is itself shorter than 8 words)
- entities: optional structured fields the project may use, e.g.
 - Zitate → {"quote": "...", "author": "..."}
Movies → {"title": "...", "year": "..."}
Omit if nothing applies.

Reply ONLY via the structured response schema. Never include explanations.

role: user → `Talk is cheap. Show me the code.` · Der Capture ist **nur das Zitat** — den `author` füllt das Modell aus eigenem Wissen.

System-Prompt ② — Anreicherung (Zitat)

`ZitateEnricher.FetchBioAsync` → zweiter LLM-Call · role: system (`ZitateEnricherPrompts`) · `Temperature 0.2` · `MaxOutputTokens 280`

You enrich a quotation for a personal knowledge tool. You are given an author and their quote. Write 2–4 factual sentences covering: who the author is (full name, life dates if known, nationality, and their role or field), and – ONLY if you genuinely know it – roughly when or in what context the quote was said or written (a year, decade, or occasion). If you do not recognise the author, reply with an empty string. Never invent facts, dates, or attributions.

role: user →

Author: Linus Torvalds
Quote: "Talk is cheap. Show me the code."

Warum ein eigener Prompt? Andere Aufgabe als Klassifikation → eigener, fokussierter Prompt. Input: nur `author` + `quote` aus den `entities` (der **Autor** wurde schon bei der Klassifikation erkannt; hier kommt die **Autor-Info** dazu). „Never invent facts“ + Leerstring-bei-Unbekannt = **Halluzinations-Stopper**.

Warum `Temperature 0.2`? Niedrige Temperatur = wenig Zufall bei der Token-Wahl → **faktentreue, reproduzierbare** Antworten statt „kreativer“ Variation — genau richtig für eine Autoren-Bio (zusammen mit „Never invent facts“). `MaxOutputTokens 280` deckelt die Länge.

Wiederfinden: semantische Suche

Nicht nur reinwerfen — auch **per Bedeutung wiederfinden**, nicht per Stichwort.

Schritt	Was passiert
Query „alles zu Docker“	→ Embedding (Mistral <code>mistral-embed</code> , 1024 Dim.)
pgvector-Suche	HNSW · Cosine-Distanz · Sub-ms bei < 1 Mio. Zeilen
Treffer	inhaltlich ähnliche Captures — auch ohne exaktes Wort

HNSW = approximativer **Nächste-Nachbarn-Index** (sub-linear schnell) · **Cosine-Distanz** = inhaltliche Ähnlichkeit über den **Winkel zwischen den Vektoren**

`GET /api/v1/captures/search?q=...` · Provider per Config tauschbar (OpenAI-kompatibel) ·
ohne Key → sauberes `503`

Tech-Stack

Schicht	Technologie
Backend	.NET 10 / C# / ASP.NET Core (LTS)
Web-UI	Blazor SSR — .NET-native, kein JS-Framework
KI-Integration	Microsoft.Extensions.AI + Ollama (lokal) / Anthropic · OpenRouter (Fallback)
Pipeline	MassTransit — In-Memory (dev) / RabbitMQ (prod)
Persistenz	PostgreSQL 17 + pgvector · EF Core 10
Deployment	Docker Compose — 6 Services, Migrations als Init-Container

Inkrementell über 5 Blöcke gebaut: Konzept → UI → Services/KI → Persistenz → Deployment.

Genutzte externe Services

● **Live-Demo:** <https://demo.flowhub.freaxnx01.ch>

Service	Rolle in FlowHub	Web
Telegram	Eingangskanal (Bot)	telegram.org
Vikunja	To-do / Projekte (Inbox, Zitate ...)	vikunja.io
Wallabag	Read-Later für Artikel / URLs	wallabag.org
paperless-ngx	Dokumenten-Management (DMS)	docs.paperless-ngx.com
OpenRouter	LLM-Gateway — Klassifikation (Gemma)	openrouter.ai
Mistral	Embeddings (1024-dim) → Suche	mistral.ai



Cloud-LLM-Adapter: **Anthropic** (Claude) · Hosting-Policy ADR 0007 (Default lokal **Ollama** geplant) · Persistenz **PostgreSQL + pgvector**.

Betrieb: Monitoring & selbst heilen

Was	Wie
Metriken	OpenTelemetry → Prometheus (<code>/metrics</code>) → Grafana
Health	<code>/health/live</code> · In-App-Integration-Health (Vikunja/Wallabag/Paperless)
Demo-Status	öffentliche Uptime-Kuma -Statusseite (<code>status.demo.flowhub...</code>) — prüft auch LLM-Erreichbarkeit
Self-Healing	Container- <code>restart</code> -Policies + Healthchecks · KeywordClassifier-Fallback bei LLM-Ausfall

Teil 2: Bauen mit KI

Erfahrung · Harness · Learnings

Der Harness – Überblick

Die KI wurde nicht ad-hoc geprompted, sondern über einen **Tool-Workflow** gesteuert:

Ebene	Werkzeug
Agent	Claude Code (interaktiv) · Codex / Copilot ergänzend
Konventionen	<code>ai-instructions</code> (base + <code>dotnet-blazor</code>) → <code>CLAUDE.md</code>
Workflows	eigene Skills: <code>/ui-*</code> , <code>/flowhub-*</code> , <code>/commit</code> , <code>/push</code>
Methode	Brainstorm → Spec → Plan → Subagent → Review
Automatisierung	<code>agent-pipeline</code> (Issue→PR) · <code>examiner-sim</code> (Grading)
Disziplin	Context-Hygiene: Logs-via-File · <code>/clear</code> -Schnitte

Nicht „Prompt rein, Code raus“ — eine Pipeline

Jeder grössere Baustein lief durch denselben strukturierten Ablauf:

Brainstorm → Spec → Plan → Subagent-Implementierung → 2-stufiges Review

1. **Brainstorming** — Design als **A/B/C-Entscheidungen** (z. B. 13 Entscheide für die Async-Pipeline), jede mit Begründung
2. **Spec + Plan** — schriftliches Design, dann **TDD-geordneter** Aufgabenplan
3. **Subagenten** — pro Task ein **frischer Kontext** (Test-First)
4. **Review ×2** — Spec-Konformität, dann Code-Qualität — bevor etwas in `main` geht

Werkzeuge: **ai-instructions** + eigene **Skills**

ai-instructions (eigenes Repo) — Konventionen als **feste Regeln**, nicht als Prompt:

- **base** (stack-agnostisch) + **Stack-Overlay** **dotnet-blazor** → daraus leitet sich **CLAUDE.md** ab
- z. B. **Coding Guidelines** (Clean Code) · **SemVer** · **Conventional Commits** · **12-Factor** (Prinzipien für Cloud-native Apps) · **TDD** — Tests werden nie nachträglich angepasst, nur damit Code grün wird

Eigene Skills (Claude-Code-Slash-Commands):

- **/ui-brainstorm** · **/ui-flow** · **/ui-build** · **/ui-review** — der **4-Phasen-UI-Workflow**
- **/flowhub-capture** · **-trriage** · **-issue** — das Produkt selbst bedienen
- **examiner-sim** · **cas-aise-grade-self-check** — **Selbstbewertung** gegen die Moodle-Bewertungskriterien

Obsidian-Vault als 2nd Brain

`vault/` — reine **Markdown**-Dateien als gemeinsamer Wissensspeicher von Mensch **und** Agent:

- Der Agent **liest** Kontext — CAS-Scope, Block-Inhalte, Entscheidungen
- ... und **schreibt** zurück — Nachbereitungen, Knowledge-Notizen, Learnings
- **Markdown**: menschen- und LLM-lesbar, git-/diff-fähig — keine Export-/API-Schicht
- Konventionen in `vault/CLAUDE.md` — Tags (`claude-generated` / `-updated`), Auto-Commit

Beispiel: der UI-Workflow

`/ui-brainstorm` → **ASCII-Wireframe** → `/ui-flow`
→ **Mermaid-Flow** → `/ui-build` → `/ui-review`

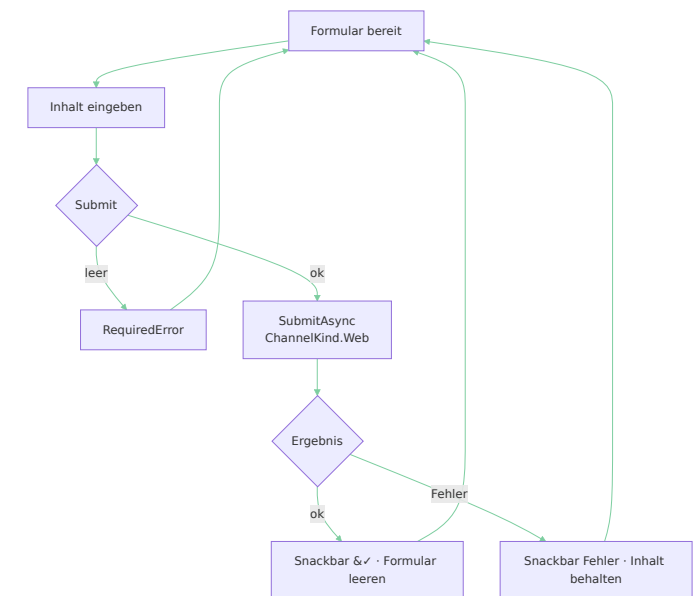
Gate pro Phase — nichts wird gebaut, bevor Wireframe **und** Flow freigegeben sind.

Phase 1 — Wireframe (`New Capture`):

```
FlowHub —
New Capture
  Content * —
    URL / Zitat / Text...
  Skill: [ — KI entscheidet ▼ ]
         [Abbrechen] [Speichern] |
```

Phase 2 — Mermaid-Flow → echtes

`docs/design/new-capture/flow.md`



Context-Hygiene

Der Kontext ist das **knappste Gut**. Zwei Disziplinen brachten am meisten:

1 · Logs via File — grösster Token-Fresser waren Console-, Test- und Build-Streams. Statt alles in den Chat: in eine **Datei** schreiben, gezielt mit `Read offset/limit` oder `grep` holen. → **~5-10× weniger Tokens** pro Debug-Session.

2 · /clear-Schnitte — Spec → `/clear` → Plan → `/clear` → Implement.

Jede Phase hinterlässt ein **Artefakt auf Disk**; der Dialog-Ballast wird verworfen.

| *Was zwischen Phasen weiterleben muss, gehört in eine **Datei** — nicht in den Chat.*

Automatisierung — KI prüft KI

`cas-aise-submission-preflight` (Multi-Agent-Check) — **Dry-Run der Moodle-Abgabe:** baut das Bundle, prüft alle Verweise, scannt auf Leaks → **Go/No-Go** vor dem Upload.

`examiner-sim` (Multi-Agent-Workflow) — baut die Abgabe-PDFs, **benotet** sie gegen die **Moodle-Bewertungskriterien** mit einem Agenten-Panel und bedient die Live-Demo.

agent-pipeline — autonome Issue → PR

Wiederverwendbarer GitHub-Actions-Workflow (`freaxnx01/agent-pipeline`); der lokale `.github/workflows/claude.yml` reicht die Arbeit nur an die Pipeline weiter.

```
Issue + Label "ai-implement"
-> claude.yml -> agent-pipeline/claude-implement.yml@main
-> Branch · commit · Claude implementiert · Draft-PR
-> Issue-Kommentar: Run-URL · PR-Link · Retry-Hinweise
```

Auslösen

- Issue mit `ai-implement` labeln; oder manuell Actions → Run workflow.
- `ubuntu-latest`, Timeout 60 min.

Retry-Policy

- `attempt` -Zähler, Reruns gedeckelt.
- Rate-Limit erreicht → **automatischer** Retry.
- `max-turns` -Erschöpfung → Kommentar im Issue, **Mensch** entscheidet.

LLM-Run (Claude)

- Auth via `CLAUDE_CODE_OAUTH_TOKEN`.
- Budget über `max-turns`.
- Claude-Code-Action meldet je Lauf **Modell + Token-Verbrauch** (Input/Output).

Wo die KI glänzt — und wo nicht

Über alle Blöcke: **100 % des Codes KI-generiert** — ~5 % brauchten menschliche Nacharbeit.

Glänzt — repetitiv & gut spezifiziert

7 `IEntityTypeConfiguration<T>`, EF-Migrations, CI-YAML;

16 Integrationstests gegen echtes PostgreSQL — **alle grün beim ersten Lauf.**

Scheitert — wo Domäne & Performance zählen

- **N+1-Blindheit** — `ListAsync` ohne `.Include`
- **CASCADE überall** — Löschen kaskadiert blind (Eltern weg → alle Kinder weg). Was erhalten bleiben muss (z. B. Audit-Trail), ist eine **menschliche Entscheidung**
- **Veraltete Versionen** — Trainingsdaten hinken neuen Releases hinterher
- **Feature-Drift** — Scope-Disziplin muss vom **Menschen** kommen

Der Smoke-Test-Moment

Die KI schrieb den ganzen Deployment-Stack. Dann lief **ein** Befehl:

`make smoke-prod` — End-to-End-Probe des laufenden Stacks.

Folgende Bugs wurden gefunden:

- `.editorconfig` fehlt im Build-Image → Build bricht mit `TreatWarningsAsErrors` ab
- Compose-Env-Casing `${EMBEDDINGS__APIKEY}` $\neq$ `Embeddings__ApiKey` → Embeddings still no-op
- Leerstring-Modellname → `AssertNotNullOrEmpty` -Crash beim Start
- Mistral lehnt das `dimensions` -Feld ab → 422

¡Al, caramba!

*KI schrieb den Code. Eine **KI-gestützte Prüfung** fand, was die KI übersah.
Der Mensch bleibt im Loop — als Reviewer.*

Fazit

KI ist ein starker Accelerator für Coding —

Logik, Boilerplate, EF-Migrations, Tests entstehen in Minuten.

Sie braucht menschliche Führung bei Architektur & Domäne —

FK-Strategie, Performance, Scope.

Der Mensch bleibt Architekt und Reviewer.

Code & Doku: github.com/freaxnx01/FlowHub-CAS-AISE · Danke — Fragen?

